

Fine-tuning LLMs for Classification Tasks

Piranavaskanthan Abeyakaran
Georgia Institute of Technology
Atlanta, GA
pabeyakaran3@gatech.edu

Rishikesh Badari
Georgia Institute of Technology
Atlanta, GA
rbadari3@gatech.edu

Jatong Su
Georgia Institute of Technology
Atlanta, GA
jsu73@gatech.edu

Abstract

This project explores how well Large Language Models (LLMs) can classify handwritten digits from the MNIST dataset. Traditionally, Convolutional Neural Networks (CNNs) are used for this task, but we explore using LLMs with eight different image encoding methods: flat and row-wise variations of pixel-value sequences, normalized pixel-value sequences, base64 encoding, and binary inputs. Our goal is to find the best encoding method to adapt LLMs for image classification, showing their potential beyond just natural language processing.

1. Introduction

Large Language Models (LLMs) have achieved impressive results in various tasks, from generating text and solving advanced algorithms. This project explores how well LLMs can adapt to classification tasks that are usually handled by architectures like Convolutional Neural Networks (CNNs). We specifically are examining how well they can perform at the MNIST handwritten digit classification task using eight image encoding strategies: both flat and row-wise formats of pixel-value sequences, normalized pixel-value sequences, base64 encoding, and binary inputs. By fine-tuning an LLM with each method, we aim to identify which image representation best helps the model interpret the images. We hope to offer insight into how LLMs handle visual data, potentially suggesting better alternatives to CNNs.

1.1. Problem Statement

Using the MNIST handwritten digit classification task as our benchmark, this project investigates which of our eight

image encoding methods enables Llama-2-7b to best preserve spatial information when converting images to text for classification. We compare four encoding types (pixel-value sequences, normalized pixel-value sequences, base64, and binary), each implemented in both flat and row-wise formats, to determine how different representations and spatial preservation affect classification performance.

1.2. Inputs and Outputs

Inputs: Our model accepts MNIST handwritten digit images encoded in eight different formats:

- **Pixel-Value Sequences:**

- **Flat:** Each 28x28 grayscale image is flattened into a single sequence of 784 pixel values (0-255).

"0 0 0 ... 84 201 243 248 14 ... 0 0"

- **Row-wise:** The 28x28 image is preserved as 28 rows of 28 pixel values each.

"0 0 0 ... 84
201 243 248 ... 167
14 89 122 ... 0"

- **Normalized Pixel-Value Sequences:**

- **Flat:** Each 28x28 grayscale image is flattened into a single sequence of 784 normalized values (0-1) by dividing each pixel value by 255.

"0.0 0.0 0.0 ... 0.329 0.788 0.953 0.973
0.055 ... 0.0 0.0"

- **Row-wise:** The 28x28 image is preserved as 28 rows of 28 normalized values each.

"0.0 0.0 0.0 ... 0.329
0.788 0.953 0.973 ... 0.655
0.055 0.349 0.478 ... 0.0"

- **Base64 Sequences:**

- **Flat:** Images are converted to bytes saved as PNG before being read and encoded into base64 format as a continuous string.

"iVBORw0KGgoAAAANSUh..."

- **Row-wise:** The images are split into 28 rows and then converted into PNG before being read and encoded into base64 and concatenated with new-lines.

"E6gEBDk...
umwFukt...
ZR1jRL0..."

- **Binary Sequences:**

- **Flat:** Images are converted to binary (0/1) by thresholding at 128, creating a sequence of 784 digits.

"0 0 0 ... 1 1 1 1 0 ... 0 0"

- **Row-wise:** The binary sequence is split into 28 rows of 28 digits each.

"0 0 0 ... 1
1 1 1 ... 0
0 0 0 ... 0"

Outputs: The encoded image is first tokenized and passed through Llama-2-7b. We then apply argmax to the model's output to determine the predicted digit. This is implemented with HuggingFace's AutoModelForSequenceClassification, which adds a classification head on top of Llama-2, and LoRA for efficient fine-tuning of specific attention layers.

1.3. Importance

Understanding how LLMs process encoded image data can change our approach to computer vision. CNNs have been the go-to choice for image tasks due to their ability to learn spatial features in images, but success in this project would suggest that LLMs, with the appropriate inputs, can also learn spatial relationships without the specialized architecture of a CNN. It can also perhaps lead to more flexible approaches where an LLM handles both visual and textual tasks.

1.4. Related Work

1.4.1 A Study of Optimizations for Fine-Tuning Large Language Models

Singh et al. [1] examine methods for fine-tuning large language models, focusing on Gradient Checkpointing, Low-Rank Adaptation (LoRA), DeepSpeed's Zero Redundancy Optimizer (ZeRO), and FlashAttention. They found that

combining ZeRO-2 with LoRA offers the best balance between memory efficiency and training time. LoRA reduces the number of trainable parameters, but it also maintains model quality — for example, a 70B parameter model can be reduced to 131M trainable parameters (about 0.19% of the original size).

From our project proposal, our TA emphasized that some optimizations affect model accuracy while others don't. Gradient Checkpointing, ZeRO, and FlashAttention are only computational optimizations that only benefit training time, whereas LoRA actually changes parameters, which can actually increase accuracy.

Based on this study, we implemented LoRA in our fine-tuning pipeline using similar hyperparameters: rank=8, alpha=16, and dropout=0.1, focusing on the attention layers query and value projection matrices. This helped us to fine-tune our Llama-2-7b model with manageable GPU requirements.

1.4.2 Text Classification via Large Language Models

Sun et al. [2] introduce Clue And Reasoning Prompting (CARP), a framework that improves LLM performance in text classification by finding clues in the input, such as keywords and other semantic relations, and using these for reasoning before the final classification. Their results show that LLMs can achieve the best performances with effective prompting.

Our project does have different goals, but it shares the idea of classification tasks. CARP focuses on text classification by using LLM reasoning abilities, whereas we explore how LLMs can classify images using various text encoding strategies. Just as the authors structured prompts to help LLMs reason, we explore how to structure visual information into text to help the LLMs learn optimally. If LLMs can classify text effectively, we believe they have potential in image classification too.

1.4.3 Large Language Models are Good Prompt Learners for Low-Shot Image Classification

Zheng et al. [3] explore how Large Language Models can improve vision models in classification tasks by providing detailed visual descriptions. They introduce LLaMP, a framework that uses LLMs to make prompts for CLIP's text encoder, linking language and vision. The core idea is that LLMs hold very descriptive data about objects, but translating this so vision and language can work together is very difficult.

Our method is similar in the goal to use LLMs for classification, but we're focus on encoding images directly as text. Like LLaMP, we're looking at the challenge in different modes of input. The authors use CLIP's text encoder,

while we try other text encoding methods to represent images. Their success in using LLMs to describe images for classification supports our exploration of LLMs as direct classifiers.

2. Technical Approach

2.1. Methodology

Our approach uses Llama-2-7b with a classification head added through HuggingFace's AutoModelForSequence-Classification. We implemented fine-tuning using Low-Rank Adaptation (LoRA) with the following configuration: Rank (r) = 8, LoRA alpha = 16, Dropout rate = 0.1, Target modules: query and value projection layers (" q_{proj} ", " k_{proj} ", " v_{proj} ", " o_{proj} ").

The training process uses mixed precision with a per-device batch size of 16 and gradient accumulation steps of 2. We train for one epoch with a learning rate of $6e-4$, with 50 warm-up steps.

For image encoding, we implement a custom encoding function for each of the eight encoding variations and apply them to the MNIST dataset.

2.2. Baseline Method

Our baseline method uses the Llama-2-7b model with an added classification head but without any fine-tuning. This is our model's normal ability to classify digit images based only on its pre-training. This provides with a reference point for measuring the effectiveness of our fine-tuning approach. However, using the current method of a classification head, the weights are initially untrained, and because of that, there is some randomness with how it predicts the output. We recognize that this may not be the best way to formulate a baseline because the model is trained to output a token sequence and is unaware of the classification head, and we address this in the next section.

2.3. Alternative Classification Approaches

In addition to using HuggingFace's classification head, we explored a more direct approach by attempting to prompt-engineer the base model to output digits directly. This involved:

- **Prompt Engineering:** We experimented with various prompt formats such as "The following is an encoded handwritten digit. Output the single digit that it represents." to encourage the model to output single-digit responses.
- **Output Parsing:** We developed methods to parse the model's natural language outputs into numerical predictions, including taking the first or last digit in the output as the prediction.

- **Challenges:** The main difficulty we encountered was the increased training time and inconsistent output format. Training took significantly longer because the model didn't stop generating tokens until the sequence ended, and it would often repeat digits over and over again.

While this approach showed promise, we ultimately found that it didn't significantly improve the model's baseline.

2.4. Evaluation Procedure and Metrics

We evaluated model performance using a comprehensive set of metrics:

- **Accuracy:** Overall classification accuracy across all digits
- **Classification Report:** Per-digit precision, recall, and F1-scores
- **Confusion Matrix:** Detailed analysis of classification errors

The evaluation procedure has the following steps. First, 10,000 encoded images from the MNIST dataset are passed through the model. The model makes predictions by applying the argmax function on its logits. These predictions are compared against the ground truth labels to find out how accurate they were. Finally, performance metrics are generated.

We evaluate both the baseline and fine-tuned models using identical procedures, so we have a direct comparison of any potential improvements. This evaluation is conducted in with gradient computation disabled for consistent results.

3. Results

3.1. Data Collection/Dataset

The standard MNIST dataset was accessed through HuggingFace's datasets library. We used the entire dataset comprising of:

- Training set: 60,000 handwritten digit images
- Test set: 10,000 handwritten digit images
- Image dimensions: 28x28 pixels
- Format: Grayscale (0-255 pixel values)

The original train/test split was maintained for consistent comparison with existing literature. Each image was pre-processed through all eight encoding variations while preserving class distributions.

3.2. Baseline Evaluation

The unmodified Llama-2-7b model with an added classification head showed the following accuracies:

Sequence Type	Flat Accuracy	Row-wise Accuracy
Pixel-Value	8.7%	11.0%
Normalized Pixel	11.0%	8.7%
Base64	9.6%	11.2%
Binary	11.1%	11.6%

The baseline confusion matrices revealed that the model tended to heavily favor predicting specific digits, suggesting it had not learned meaningful features from the encoded inputs before fine-tuning.

3.3. Proposed Method Results

After fine-tuning with LoRA, we observed the following accuracies:

Sequence Type	Flat Accuracy	Row-wise Accuracy
Pixel-Value	98.7%	99.1%
Normalized Pixel	76.9%	68%
Base64	60.2%	95.9%
Binary	98.5%	98.8%

The implementation of each encoding method presented surprising results:

- **Pixel-Value Sequences:**

- Flat: Only slightly higher than binary sequence performance despite more information in the input
- Row-wise: Highest accuracy most likely due to spatial and pixel information being preserved

- **Normalized Pixel-Value Sequences:**

- Flat: Performed significantly worse than pixel-value and binary sequences
- Row-wise: Performed worse than flat version despite having more spatial information

- **Base64 Sequences:**

- Flat: Showed limited improvement with compact representation, suggesting obscured spatial relationships
- Row-wise: Huge jump in accuracy once some spatial information is preserved

- **Binary Sequences:**

- Flat: Achieved close to pixel-value accuracy, suggesting the positioning matters more than value
- Row-wise: Similar increase as pixel value sequences

These results suggest that preserving spatial relationships in the encoding is more important than maintaining grayscale precision, as we’ve seen by binary thresholding’s strong performance compared to the more complex base64 encoding.

3.4. Analysis and Figures

These confusion matrices provide insight into how the model behaves across encoding methods:

3.4.1 Pixel-Value Sequences

Flat Format

970	0	0	0	2	1	2	2	2	1
0	1129	2	1	0	2	1	0	0	0
2	3	1005	6	0	0	2	10	3	1
0	0	2	1001	0	3	0	3	0	1
1	0	0	0	972	0	3	0	1	5
0	0	0	5	0	883	1	1	0	2
4	3	1	0	2	2	945	0	1	0
2	1	11	1	0	0	0	1009	0	4
1	0	1	1	0	0	0	2	967	2
3	0	0	1	4	1	0	4	7	989

Row-wise Format

974	0	1	0	1	0	2	2	0	0
0	1131	1	3	0	0	0	0	0	0
2	2	1008	5	1	1	0	10	3	0
0	0	1	1003	0	2	0	1	3	0
0	0	0	0	974	0	2	1	2	3
0	0	0	3	0	886	1	1	0	1
3	2	0	1	1	1	948	0	1	1
1	3	7	1	0	0	0	1013	0	3
1	0	1	0	0	0	0	0	971	1
0	0	0	0	6	1	0	0	1	1001

With pixel-value encodings, the model occasionally confuses digits with similar shapes, such as "3" and "8" but is generally successful. The flat format’s success suggests that the LLM can learn spatial relationships indirectly from the positions of pixel values, even without explicit spatial structure. However, the lack of row-wise structure likely limits the model’s ability to fully capture local pixel patterns, leading to slight confusion in similar digits.

3.4.2 Normalized Pixel-Value Sequences

Flat Format

672	3	43	13	7	21	10	13	181	17
12	1082	1	4	9	4	16	2	0	5
108	4	604	212	3	7	7	15	64	8
52	1	142	659	1	13	1	52	68	21
10	54	1	3	857	1	14	11	9	22
28	2	8	29	4	705	22	44	45	5
6	10	15	0	4	17	894	3	8	1
2	1	8	74	6	14	0	830	9	84
153	2	41	62	22	49	15	29	560	41
20	3	5	9	8	12	0	69	59	824

Row-wise Format

540	16	81	84	13	25	22	20	159	20
20	1055	1	3	35	1	11	0	4	5
127	7	562	226	1	12	11	26	51	9
85	2	143	576	4	35	3	46	84	32
11	127	2	0	756	10	12	29	14	21
54	4	22	88	25	482	22	107	75	13
26	11	27	3	6	8	873	1	3	0
7	2	5	56	33	17	0	752	19	137
153	9	40	157	35	66	15	21	427	51
17	4	4	16	14	4	0	108	62	780

Despite it being common practice to normalize feature vectors when training models, our model performs far worse with normalized pixel-value encodings than with raw pixel-value encodings. This could be because normalization makes it harder for the model to distinguish between intensity of pixels, as inputs are treated as tokens and not floats, leading to inaccurate outputs.

3.4.3 Base64 Sequences

Flat Format

628	2	49	75	16	7	35	20	129	19
3	1070	9	16	11	2	12	7	1	4
217	9	441	136	23	25	53	23	88	17
138	12	73	549	27	71	22	22	62	34
69	19	19	32	572	3	47	42	68	111
75	10	52	239	25	317	27	54	63	30
85	11	43	44	34	7	545	12	100	77
1	12	16	8	24	13	14	822	5	113
229	10	39	101	34	19	60	12	432	38
24	7	9	24	34	5	53	173	40	640

Row-wise Format

953	0	0	0	6	1	1	2	4	13
2	1122	4	4	0	1	1	1	0	0
2	1	993	4	1	1	7	11	10	2
0	0	13	954	0	17	0	13	6	7
3	1	2	1	934	1	7	1	2	30
4	1	0	21	1	844	3	5	9	4
7	2	2	0	5	2	937	0	2	1
4	4	34	11	1	0	0	962	4	8
5	0	1	7	3	6	1	2	944	5
5	2	0	4	15	4	0	10	21	948

The model significantly improves when switching from flat base64 encoding to row-wise base64 encoding. We hypothesize that this is because far more information is added to the file when the entire image is converted into a PNG than when a row is converted into a PNG due to optimizations necessary for compression. As a result, the row-wise base64 encodings represent simpler values that more directly reflect the actual pixel information in the image.

3.4.4 Binary Sequences

Flat Format

975	0	0	0	0	1	2	1	0	1
0	1128	2	0	0	2	3	0	0	0
5	2	1004	5	0	0	2	8	5	1
0	0	1	1001	0	3	0	1	1	3
0	0	0	0	968	0	2	1	0	11
1	0	0	8	0	874	4	2	2	1
2	2	0	1	1	2	949	0	1	0
2	1	19	5	0	0	0	998	0	3
2	0	2	1	3	0	0	2	963	1
1	0	0	4	4	1	0	4	6	989

Row-wise Format

976	0	0	0	0	0	1	3	0	0
0	1127	3	1	0	0	4	0	0	0
2	2	1013	4	0	0	0	9	2	0
0	0	2	999	0	6	0	1	2	0
0	0	0	0	968	0	2	2	0	10
1	0	0	5	0	878	2	1	2	3
5	2	1	1	1	2	945	0	1	0
0	3	12	2	0	0	0	1009	0	2
0	0	1	2	0	1	0	1	967	2
4	0	0	0	2	0	0	5	1	997

With binary encodings, the model performs almost well as pixel-value encodings, just confusing "8"s for "3"s. This shows that preserving spatial relationships, even in a simplified binary representation, is critical for LLM-based image classification, while grayscale precision appears less

essential. The slight edge of row-wise encoding suggests that explicitly maintaining spatial structure benefits the model’s ability to capture local patterns.

4. Conclusion

Our investigation into using LLMs for image classification through various encoding methods has yielded several key insights. The strongest performing method was row-wise pixel value sequence encoding (99.1% accuracy), closely followed by row-wise binary sequences (98.8%), while base64 encoding showed limited effectiveness (60.2%). These results suggest that simpler, more direct representations of spatial information can be effectively processed by LLMs, challenging the assumption that specialized architectures like CNNs are necessary for image classification tasks.

We encountered several significant findings:

- Binary encoding’s success suggests that preserving basic spatial relationships is more important than maintaining fine-grained pixel values
- The poor performance of base64 encoding indicates that LLMs struggle with highly compressed, abstract representations of image data
- The comparable performance between binary and pixel-value sequences suggests that LLMs can effectively process both simplified and detailed input formats

Future work could explore several promising directions:

- Investigating how these encoding methods perform on more complex image classification tasks beyond MNIST
- Exploring hybrid approaches that combine the benefits of LLMs and CNNs
- Developing more sophisticated prompt engineering techniques to improve classification without requiring a classification head
- Analyzing attention patterns to better understand how LLMs process spatial information in different encoding formats

These results demonstrate that LLMs can effectively handle image classification tasks when provided with appropriately encoded inputs, suggesting broader applications for these models beyond pure language tasks. The success of simpler encoding methods also indicates that future work in multimodal AI might benefit from focusing on clear, spatially-preserved representations rather than complex encodings.

5. Team Contributions

Piranavaskanthan Abeyakaran

1. Implemented pixel-value and normalized pixel-value encodings
2. Analyzed results for pixel-value sequences
3. Wrote Introduction and Technical Approach sections

Rishikesh Badari

1. Implemented base64 and row-wise base64 encodings
2. Analyzed results for base64 sequences
3. Wrote Related Work and Methodology sections

Jatong Su

1. Implemented binary and row-wise binary encodings
2. Analyzed results for binary sequences
3. Wrote Results and Conclusion sections

References

- [1] Arjun Singh, Nikhil Pandey, Anup Shirgaonkar, Pavan Manoj, and Vijay Aski. A study of optimizations for fine-tuning large language models. *arXiv preprint arXiv:2406.02290*, 2024. [2](#)
- [2] Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. Text classification via large language models. *arXiv preprint arXiv:2305.08377*, 2023. [2](#)
- [3] Zhaoheng Zheng, Jingmin Wei, Xuefeng Hu, Haidong Zhu, and Ram Nevatia. Large language models are good prompt learners for low-shot image classification. *arXiv preprint arXiv:2312.04076*, 2023. CVPR 2024. [2](#)